

Video:1

Introduction to [#RestAssured](#):

As part of this video will discuss about:

- What is Rest-Assured [#API](#)/Library?
- Rest-Assured is a [#DSL](#) (Domain Specific Language) based Java API/Library to test the Restful APIs.
- It is an open-source Library.
- Basically used to Automate the API testing.
- With the help of Rest-Assured we can test:
- Status codes of your response.
- Status messages
- [#Headers](#)
- Body etc...
- **Advantages of Rest-Assured Library.**
- Ease of use: Simple and concise syntax makes writing the test cases straightforward.
- Readability: Tests are more readable compared to raw Java code, even for unfamiliar users.
- Versatility: Supports all common [#HTTP](#) methods and various API features.
- Assertion Power: Offers a wide range of options for verifying the response data.
- Integrates with testing frameworks: Seamlessly works with popular unit testing frameworks like JUnit and TestNG.
- **Disadvantages of Rest-Assured Library.**
- Java-Specific: Only works with Java Programming Language.
- Initial Learning curve: Getting comfortable with features and conventions might require some effort.

What is DSL? It provides a specialized language tailored for REST API testing, making test writing more efficient and expressive. Example Code: `given().baseUrl("https://api.example.com`

`").when().get("/users").then().statusCode(200).body("name", hasItem("Tom"))`); Official Website: <https://rest-assured.io/>

Video=2:

RestAssured (C):

- Main entry point for using Rest-Assured.
- Provides methods to set global configurations such as default base [#URI](#), base Path and Port etc...
- RestAssured.baseURI = "<https://api.example.com>

";

- RestAssured.port = 800;

RequestSpecification (I):

- Interface that represents the specification for a request.
- It will allow you to set headers, query parameters, body and authentication details etc...
- `given().param("key1", "value1").header("Content-Type", "application/json")`

Response (I):

- Represents the response obtained after making the request.
- Contains details like status code, response body and headers etc...
- Provides methods to extract and validate different parts of the response.

Filter (I):

- Interface used for specifying filters that can modify the request or response.
- Filters can be used to logging, error handling, or other customizations.

JSONPath (C):

- Class for parsing the JSON response.
- Allow easy extraction of data from the response using [#JSONPath](#).

XMLPath (C):

- Class for parsing the XML response.
- Allow easy extraction of data from the response using [#XMLPath](#).

Request/Response Logging:

- Logging the details of the request and response can be crucial for debugging.
- `log.all()` -- Logs all the details of the request of response.
- `log.ifValidationsFails()` -- Log details only if validation fails.

Authentication:

- Classes and method for handling authentication such as
- basic authentication
- OAuth authentication
- OAuth2 authentication etc...

Matchers:

- Class that provides various static methods for performing assertions on the response.
- Used to validate status codes, response body content and headers etc...
- `equalsTo()`, `hasItems()`, and `containsString()` etc...
- `assertThat().body("name", equalsTo("Tom"))`

Query Parameters:

- Adding Query Parameters to the request.
- `.queryParams("paramName", "paramValue")`

Path Parameters:

- Adding Path Parameters to the request.
- `.pathParam("paramName", "paramValue")`

Headers:

- Setting headers for request.
- `.header("headerName", "headerValue")`

URI Building:

- Building the complete URI for the request.
- `basePath(String path)` -- Sets the base path for the request.
- `baseUri(String uri)` -- Sets the base URI for the request.

Methods: ----- `given()` -- Starts building a request specification. `when()` -- Executes the request. `then()` -- Validates the response using matchers. `get()`, `post()`, `put()`, `delete()`, `patch()` -- Perform specific HTTP methods. `body()` -- Sets or retrieves the request or response body. `header()` -- Sets or retrieves headers. `cookie()` -- Sets or retrieves cookies. `statusCode()` -- Get the status code of the response. `statusLine()` -- Get the status line of the response. `assertThat()` -- Asserts response details using matchers.

Video=7:

What is JSON?: -JSON -- JavaScript Object Notation.

- It is a lightweight data interchange format that is easy for humans to read and write, and easy for machines to parse and generate.

- It is commonly used for transmitting data between server and a web application, as well as for configuration files. Basic Structure:

- JSON data is represented as a collection of Key-Value pairs.
- Data is stored in Key-Value pairs, where the Key (a String) is

followed by a colon, and then the Value. Data Types:

- Strings : A sequence of characters enclosed in double quotes.
- Numbers: Integer or Floating-Point numbers.
- Objects : An unordered collection of Key-Value pairs, enclosed in

curly braces i.e. {}

- Arrays: An ordered list of values, enclosed in square brackets

i.e. []

- Boolean : true or false
- null : Represents an empty value or absence of a value

Example: { "name" : "Tom", "age" : 30, "isStudent" : false, "courses" : ["Java", "Rest Assured", "Selenium"] } Keys -- name, age, isStudent and courses. Values -- Tom, 30, false, ["Java", "Rest Assured", "Selenium"]. Objects:

- Objects in JSON are enclosed in curly braces i.e. {}.
- Each Key-Value pair in an object is separated by a comma.

Arrays:

- Arrays in JSON are ordered lists of values and are enclosed by

square bracket i.e []

- Elements in an array can be of any JSON data type, including other

arrays or objects. Nested Structure:

- JSON can have nested structures, where objects or arrays can be

values of other arrays or objects. Nested Structure Example: { "person": { "name":"Tom", "age" : 30, "address": { "city":"New York", "country":"USA" } }, "hobbies":["Reading", "Painting"] }

Encoding and Decoding: JSON is used to represent data in a text format. When transmitting the data, it needs to be encoded in JSON string. On the receiving end, this JSON string is then decoded back into a data structure in the appropriate Programming language. JSON is widely used due to its simplicity, readability, and ease of use in various Programming languages. It has become a standard for data exchange between web servers and clients.

Video-8:

What is JSON Path:

- JSONPath is a query language used to navigate and query JSON data structures.
- It provides a way to express paths to elements within a JSON document.
- It allows you to extract specific data or filter information based on certain criteria.
- JSONPath is similar in concept to XPath for XML documents.

understand JSON Structure: Before using JSONPath, familiarize yourself with the structure of the JSON document you are working with. Identify Key-Value pairs, arrays and nested structures etc...

Basic Syntax:

- JSONPath expressions use a similar syntax to XPath.
- They start with the root elements (\$) followed by a sequence of

Keys, arrays or wildcards. Examples: \$ -- Represents the root of the JSON document. \$.key -- Access the the value for the specified key. \$.array[0] -- Access the first element of the Array. Child Operator (.):

- Use a dot (.) operator to navigate to child elements.

Recursive Descent (..):

- The double-dot (..) notation allows you to descend into all nested elements, regardless of their depth. Wildcard (*):

- The asterisk (*) is a wildcard character that matches any key or array index at the current level. Filtering:

- You can use filters to extract specific elements based on conditions.

Array Slices([start:end]):

- Use array slices to retrieve a range of elements from an array.

Script Expressions('(@)'):

- Script expressions allow more compiled filtering using Programming

like expressions. Functions:

- Some JSONPath implementations support functions for more advanced querying. How to do that and where to do that? <https://jsonpath.com/>

```
{ "bookstore": { "name": "Tech Book Emporium", "location": "Silicon Valley", "books": [ { "id": 1, "title": "The Art of Programming", "author": "John Coder", "price": 29.99, "categories": ["Programming", "Computer Science"] }, { "id": 2, "title": "Web Development 101", "author": "Jane Coder", "price": 19.99, "categories": ["Web Development", "JavaScript"] }, { "id": 3, "title": "Data Science Essentials", "author": "Alice Analyst", "price": 34.99, "categories": ["Data Science", "Statistics", "JavaScript"] } ] } }
```

Examples: 1. Root of the JSON document? -- \$

2. Child Operator:

- Access the name of the bookstore?

`$.bookstore.name`

- Access the title of the first book?

`$.bookstore.books[0].title`

3. Recursive Descent:

- Retrieve all author names under the bookstore?

`$.bookstore..author`

4. Wildcard:

- Retrieve categories of all books?

`$.bookstore.books[*].categories`

5. Filtering:

- Retrieve books with a price less than 30?

`$.bookstore.books[?(@.price lessthan 30)]`

6. Array Slices:

- Retrieve the second and third books?

`$.bookstore.books[1:3]`

7. Script Expressions:

- Retrieve books with the categories "JavaScript"?

`$.bookstore.books[?(@.categories.includes('JavaScript'))]`

Testing the Api through the Rest-assured:

1>Create a java maven project of archetype 1.0

2>Copy the rest assured dependency from the Mvn repository and in the dependency remove the Scope to write and check the api through out the project.

<dependency>

```
<groupId>io.rest-assured</groupId>
```

```
<artifactId>rest-assured</artifactId>
```

```
<version>5.1.1</version>
```

```
<scope>test</scope>
```

 → Remove it otherwise write the api testing code under the `src/test/java` only

```
</dependency>
```

3>For the general take the 1 fake api website for testing the api → <https://fakestoreapi.com/products>

4>There are 3 ways to write the code for checking the get request

5>User may use any of the code for getting the output

```
public class Rest1 {
```

```
public static void main(String[] args) {
```

Way 1

```
RestAssured.baseURI="https://fakestoreapi.com/products";
```

```
RequestSpecification requestspec=RestAssured.given();
```

```
Response response=requestspec.get();
```

```
System.out.println(response.prettyPrint());
```

way2

```
RestAssured.baseURI="https://fakestoreapi.com/products";
```

```
RequestSpecification reqspec=RestAssured.given();
```

```
Response response=reqspec.request(Method.GET);
```

```
System.out.println(response.prettyPrint());
```

way3

```
RestAssured.given().when().get("https://fakestoreapi.com/products").then().log().all();
```

```
}
```

```
}
```

6>This above code should be runned as java code and you will get the output of the website.

Video=10

1. Use the resource -- <https://fakestoreapi.com/products/1>

2. Specify the HTTP method (i.e. GET method)

3. Send the Request to the Server.
4. Get the Response from the Server.
5. Verify the Response Code. Status Codes:
 - 1xx -- Response is informational.
 - 2xx -- Successful response.
 - 3xx -- need more information to fulfil the request.
 - 4xx -- Bad syntax and the request cannot be completed.
 - 5xx -- Server side failures.

Validation of the rest api through the Testng and rest-assured:

Video-10: Validating the status code of the Api

1> Create a java maven project of archetype 1.0

2> Copy the rest assured dependency and testng from the Mvn repository and in the dependency remove the Scope to write and check the api through out the project.

<dependency>

<groupId>io.rest-assured</groupId>

<artifactId>rest-assured</artifactId>

<version>5.1.1</version>

<scope>test</scope> → Remove it other wise write the api testing code under the src/test/java only

</dependency>

<dependency>

<groupId>org.testng</groupId>

<artifactId>testng</artifactId>

<version>7.10.0</version>

<scope>test</scope> → Remove it other wise write the api testing code under the src/test/java only

</dependency>

3> For the general take the 1 fake api website for testing the api → <https://fakestoreapi.com/products>

4> There are 2 ways to write the code for checking the get request

5> User may use any of the code for getting the output


```

public class Rest1 {

    @Test

    public void Statusvalidator() {

        //Way1

        RestAssured.baseURI="https://fakestoreapi.com/products";

        RequestSpecification reqspec=RestAssured.given();

        Response res=reqspec.get();

        int statuscode=res.getStatusCode();

        Assert.assertEquals(statuscode, 200);

        //Way2

        RestAssured.given().when().get("https://fakestoreapi.com/products").then().statusCode(400);

    }

}

```

6>You will get the response code and it will compare with the given number if its matched then it will be passed else its fails.

7>This Gherkin have the inbuilt Assertion hence we will use the assertion statuscode as direct.

```
RestAssured.given().when().get("https://fakestoreapi.com/products").then().statusCode(400);
```

Video-11:Printing and Validating Headers

```

public class Rest1 {

    @Test

    public void printAllheaders() {

        //Way1

        RestAssured.baseURI="https://fakestoreapi.com/products";

        RequestSpecification reqspec=RestAssured.given();

        Response res=reqspec.get();

        Headers headers=res.headers();

        for(Header h:headers) {

            System.out.println(h.getName()+"--->" +h.getValue());}

        //Way2
    }

}

```

```

        RestAssured.given().when().get("https://fakestoreapi.com/products").then().log().headers
    );
}

@Test
public void printspecificheader() {
    //Way1
    RestAssured.baseURI="https://fakestoreapi.com/products/1";
    RequestSpecification reqspec=RestAssured.given();
    Response response=reqspec.get();
    System.out.println("Printing the specific header details");
    System.out.println(response.getHeader("Content-Type"));
    System.out.println(response.getHeader("Connection"));

    //Way2
    Response response12=RestAssured.given()
    .when().get("https://fakestoreapi.com/products/1");
    System.out.println("Printing the specific header details");
    System.out.println(response.getHeader("Content-Type"));
    System.out.println(response.getHeader("Connection"));
}

@Test
public void validateSpecific_server() {
    //Way1
    RestAssured.baseURI="https://fakestoreapi.com/products/1";
    RequestSpecification reqspec=RestAssured.given();
    Response response=reqspec.get();
    String Content_type=response.getHeader("Content-Type");
    String Connection=response.getHeader("Connection");
    Assert.assertEquals(Content_type, "application/json; charset=utf-8");
    Assert.assertEquals(Connection, "keep-alive");

    //Way2
    RestAssured.given()

```

```

        .when()

        .get("https://fakestoreapi.com/products/1").

        then()

        .header("Connection","keep-alive")

        .header("Content-type","application/json; charset=utf-8");

    }

}

```

Video-12:Printing and Validating the Json Response body

```

public class ResponseJsonbody {

    @Test

    public void Readjsonbody() {

        //Way 1

        RestAssured.baseURI="https://fakestoreapi.com/products/1";

        RequestSpecification requestspec=RestAssured.given();

        Response response=requestspec.get();

        String response_body=response.body().asString();

        System.out.println(response_body);

        //way2

        RestAssured.given()

            .when()

            .get("https://fakestoreapi.com/products/1")

            .then()

            .log()

            .body();

    }

    @Test

    public void readjsonresponsestringvalidate() {

        //Way 1

        RestAssured.baseURI="https://fakestoreapi.com/products";

        RequestSpecification reqspec=RestAssured.given();

        Response response=reqspec.get();
    }
}

```

```

String responsebody=response.asString();

Assert.assertTrue(responsebody.contains("men's clothing"));

Assert.assertTrue(responsebody.contains("title"));

//Way2

String
Responsebody1=RestAssured.given().when().get("https://fakestoreapi.com/products/1")

.getBody().asString();

Assert.assertTrue(Responsebody1.contains("image"));

}

@Test

public void readjsonresponseandvalidate() {

    //Way1

    RestAssured.baseURI="https://fakestoreapi.com/products/1";

    RequestSpecification reqspec=RestAssured.given();

    Response response=reqspec.get();

    JsonPath jsonpath=response.jsonPath();

    String category=jsonpath.getString("category");

    String rate=jsonpath.getString("rating.rate");

    System.out.println("The category is : "+category);

    System.out.println("The rate is : "+rate);

    Assert.assertEquals(rate,"3.9");

    Assert.assertEquals(category,"men's clothing");

    //Way2

    //@@@@rating.rate and rating.count==>Json path notations==>id=1 as per the url//

    //@@@@rate under the rating will be fetched by the rating.rate //

    //@@@@Matchers class is used for Accurate comparison//

    RestAssured.given().when().get("https://fakestoreapi.com/products/1")

    .then().body("rating.count", Matchers.equalTo(120))

    .body("image",Matchers.equalTo("https://fakestoreapi.com/img/81fPKd-

2AYL._AC_SL1500_t.png"));

```

```
}  
  
}
```

Video-13:

Authentication: Authentication is the process of verifying the identity of a user, system, or entity trying to access a particular resource to perform an action. The goal of authentication is to ensure that the entity attempting access is who it claims to be. Authentication usually involves presenting credentials, such as username and password, and verifying them against a stored record. Example: Consider a scenario where a user logs into an email account. The system prompts the user to enter a username and password. If the entered credentials match the stored credentials for that user, then the system authenticates the user, granting access to the email account.

Authorization: Authorization, on the other hand, is the process of determining the actions or resources that an authenticated user or entity is allowed to access. Once user's identity is verified through authentication, authorization ensures that the authenticated entity has the necessary permissions to perform specific operations or access certain resources. Example: Using the same email, after a user successfully authenticates, the system needs to decide what actions the user is allowed to perform.

- A regular user may be authorized to read and send emails.
- An Admin user may be authorized to perform additional actions like

managing the user accounts or system settings. Key Differences: Focus:

- Authentication: Focuses on verifying the identity of the user.
- Authorization: Focuses on determining what actions or resources

an authenticated user is allowed to access. So first Authentication happens and then Authorization happens. Purpose:

- Authentication: Establishes trust by confirming the user's identity.
- Authorization: Controls access and permissions based on the authenticated

user's identity. Verification vs Permission:

- Authentication: Verifies the credentials provided by the user.
- Authorization: Grants or denies access based on the permissions

associated with the authenticated identity. Example:

- Authentication: When a user logs with a user name and password, the system checks if the credentials are valid, verifying the user identity.
- Authorization: Once authenticated, the system checks the user's role

(i.e. regular user, admin user) and grants appropriate permissions. A regular user has only read access, while an admin may have read and write access.

Video-14:Authentication and its types

Authentication in REST API? Authentication is the process of verifying the identity of the user, system, or application attempting to access or perform an action. In the context of REST APIs, authentication is essential to ensure that only authorized entities can access or manipulate resources. Various authentication mechanisms exist to secure REST APIs. Why different types of Authentications? Because, security requirements and use cases can vary significantly across different applications, systems, and scenarios. The choice of authentication method depends on factors such as security needs, ease of implementation, the nature of application, and user experience considerations. Different types of Authentications: Basic Authentication:

- Sends credentials (username and password) in the request headers.
- Uses Base64 encoding for credentials security.
- Vulnerable to eavesdropping if not used over HTTPS.

Digest Authentication:

- Challenges the client with a unique value.
- Requires the client to hash the credentials, challenge, and other data.
- More secure than Basic authentication as it doesn't send plain text

password. Preemptive Authentication:

- Proactively sends credentials with every request, regardless of server challenges.
- Useful in scenarios where the server may not always challenge the client

but expects credentials with each request. Bearer Token Authentication:

- Involves sending a token in the Authentication header.
- Commonly used token-based authentication systems like OAuth 2.0.
- The token is usually obtained through a separate authentication process.

OAuth 1.0 Authentication:

- Requires the exchange of credentials for access tokens and secret.
- Uses a combination of consumer key, consumer secret, access token and access token secret.
- More complex than OAuth 2.0 and less widely adopted.

OAuth 2.0 Authentication:

- Involves obtaining an access token to access the protected resources.
- Supports different grant types such as Authorization code, Implicit,

Resource Owner password credentials and client credentials.

- More streamlined and widely adopted when compared to OAuth 1.0

API Key Authentication:

- Involves sending an API key in the request header.
- Commonly used for machine-to-machine communication.
- Provides a simple way to identify and authorize API clients.

JWT (JSON Web Token) Authentication:

- Uses a compact, URL-safe means of representing claims between 2 parties.
- Typically used to token-based authentication and authorization.
- Can contain information about the user, expiration, and other claims.

Certificate-Based Authentication:

- Involves exchanging digital certificates to authenticate clients or servers.
- Provides a higher level of security compared to simple username/password mechanisms.
- Commonly used in enterprise environments.

OpenID Connect:

- Built on top of OAuth 2.0, provides a standard for authentication.
- Enables clients to verify identity of the end-user and to obtain

basic profile information.

Video-15: Basic Authentication

Basic Authentication in Rest-Assured:

- Basic authentication is a simple authentication scheme built into the

HTTP protocol.

- It is often used to secure your RESTful APIs.
- In Basic authentication, the client includes a username and password in

the request header using the "Authorization" field.

- The credentials are typically Base64-encoded, although it's Important

to note that Base64 encoding is not a secured method for protecting sensitive information. How it works:

- The client includes the username and password in the HTTP request header.
- The credentials are Base64-encoded and combined with the word "Basic"

followed by a space.

- The resulting string is added to the "Authorization" header of the HTTP

request.

- Limitations of Basic authentication is that it sends the credentials with every request, which can be a security risk.

- Then the terms "preemptive" and "non-preemptive" come into play.

In preemptive authentication, the client send the authentication information with the Initial request without receiving the HTTP 401 unauthorized response for the server. In non-preemptive authentication, the client only sends the authentication information after receiving the HTTP 401 response from the server.

Example: Open the postman or the HTTPie app then use the link and request for the get method:

`https://postman-echo.com/basic-auth` → You will get the error of **401 Unauthorized** → Since we are not providing any credentials.

Use the basic auth as the Auth and provide the credential which are given below and send the get request.

Username= postman and **Password=** password

→ On sending the request with the Basic auth you will get the code and message as **200 OK**

In RestAssured:

```
public class BasicAuthinRestApi {  
  
    @Test  
  
    public void Basicauth() {  
  
        RestAssured.baseURI="https://postman-echo.com/basic-auth";  
        RequestSpecification requestspec=RestAssured.given();  
        Response response=requestspec.get();  
        System.out.println(response.getBody().asPrettyString());  
    }  
}
```

From the above you will get the **401 unauthorized** error since we are not providing the basic auth credentials.

On passing the credentials to the rest assured through RequestSpecification you will get the response as {

```
    "authenticated": true  
}
```

The program with passing the basic authentication:

```
public class BasicAuthinRestApi {
```



```

    @Test

    public void Basicauth() {

        RestAssured.baseURI="https://postman-echo.com/basic-auth";

        RequestSpecification requestspec=RestAssured.given();

        requestspec.auth().basic("postman", "password");

        Response response=requestspec.get();

        System.out.println(response.getBody().asPrettyString());

    }

}

```

➔The above output is generated in the format of json for using the assertion we need to convert it from the json to string and then we have use the assertion.

```

JsonPath jsonpath=response.jsonPath();

String auth=jsonpath.getString("authenticated");

```

The program after converting the Json to string and inserting the assertion:

```

public class BasicAuthinRestApi {

    @Test

    public void Basicauth() {

        //Way1

        RestAssured.baseURI="https://postman-echo.com/basic-auth";

        RequestSpecification requestspec=RestAssured.given();

        requestspec.auth().basic("postman", "password");

        Response response=requestspec.get();

        System.out.println(response.getBody().asPrettyString());

        JsonPath jsonpath=response.jsonPath();

        String auth=jsonpath.getString("authenticated");

        Assert.assertEquals(auth, "true");

    }

}

//Way2

RestAssured.given().

auth().basic("postman", "password")

```

```
.when().get("https://postman-echo.com/basic-auth")
.then().statusCode(200)

.body("authenticated",Matchers.equalTo(true));
```

Note: true is Boolean value hence in way2 there are no double quotes.

→ Your test case will be passed

Video-16:

- **Preemptive authentication** is a method where the client sends authentication credentials with every request without waiting for the server to challenge it.
- In the context of Basic Authentication, preemptive authentication means

that the client includes the necessary credentials in the request header from the very beginning, without being prompted by the server.- Preemptive authentication is a method where the client sends authentication credentials with every request without waiting for the server to challenge it.

- In the context of Basic Authentication, preemptive authentication means that the client includes the necessary credentials in the request header from the very beginning, without being prompted by the server.

```
public class PreemptiveAuthinRestApi{
    @Test
    public void Preemptive() {
        //Way1
        RestAssured.baseURI="https://postman-echo.com/basic-auth";
        RequestSpecification reqspec=RestAssured.given();
        reqspec.auth().preemptive().basic("postman", "password");
        Response response=reqspec.get();
        System.out.println(response.getBody().asPrettyString());
        JsonPath jsonpath=response.jsonPath();
        String auth=jsonpath.getString("authenticated");
        Assert.assertEquals(auth, "true");
        //Way2
        RestAssured.given().auth().preemptive().basic("postman",
"password")
        .when().get("https://postman-echo.com/basic-auth").then()
        .statusCode(200).body("authenticated",Matchers.equalTo(true));
    }
}
```

→ 200OK → Status code → Passed

→ The Matchers class is used for matching the values.

Digest Authentication:

17. Mastering Digest Authentication in Rest Assured: A Complete Guide | API Testing Tutorial

- Digest Authentication is another HTTP-Based authentication mechanism,

like Basic Authentication.

- However, it is more secured than Basic Authentication, where the server

challenges the client with a nonce(number used once), and client responds with a hash of the nonce and other information.

- It doesn't transmit the actual password over the network.

Brief overview of how Digest Authentication works: 1. The server sends a challenge to the client, including a nonce and other parameters. 2. The client calculates a hash of the password, the nonce, and other data, and sends it back to the server. 3. The server verifies the hash and allows access to the client.

```
public class DigestAuthentication {  
  
    @Test  
  
    public void DigestAuthentication() {  
  
        //Way1  
  
        RestAssured.baseURI="https://postman-echo.com/digest-auth";  
        RequestSpecification reqspec=RestAssured.given();  
        reqspec.auth().digest("postman","password");  
        Response response=reqspec.get();  
        System.out.println(response.getBody().asPrettyString());  
        JsonPath jsonpath=response.jsonPath();  
        String auth=jsonpath.getString("authenticated");  
        Assert.assertEquals(auth, "true");  
        Assert.assertEquals(response.statusCode(), 200);  
  
        //Way2  
  
        RestAssured.given().auth().digest("postman", "password")  
        .when().get("https://postman-echo.com/digest-auth").then()  
        .statusCode(200).body("authenticated",Matchers.equalTo(true));  
  
    }  
}
```

POJO CLASS:

What is POJO and POJO Class? POJO -- Plain Old Java Object Definition of POJO:

- A simple Java class that encapsulates fields, provides setters and getters,

and follows basic Java conventions.

- Contains only data, without business logic or framework-specific

annotations. Characteristics:

- No external dependencies or framework annotations.
- Facilitates clear representation of data structures.

POJO Class: Definition of POJO Class:

- A Java class that adheres to the principles of POJO.
- Typically used to represent data structures in a simple and

readable manner. Characteristics:

• Consists of private fields with corresponding getter and setter methods.

- Encourages encapsulation and modularity.

Use in Rest-Assured: Request Payload:

- In Rest-Assured, POJOs are often used to model request payloads for

POST or PUT requests.

- Simplifies the construction of JSON or XML request bodies.

Response Payload:

- POJOs aid in the deserialization of API responses.
- Allows easy conversion of JSON/XML response bodies into JAVA objects

for manipulation.

```
public class Pojo {  
    private String name;  
    private int id;  
    private int age;  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public int getId() {
```

```

        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public Pojo(String name, int id, int age) {
        super();
        this.name = name;
        this.id = id;
        this.age = age;
    }
}

}



---


public class Pojo_Example {
    public static void main(String[] args) {
        Pojo p=new Pojo("A",1,20);
        System.out.println("Age is "+p.getAge());
        p.setAge(30);
        System.out.println("Modified age is "+p.getAge());
    }
}

```

Serialization and Deserialization:

- Serialization and deserialization are Important concepts in the context

of data transmission, storage, and communication between different systems.

- In the context of REST APIs and tools like Rest-Assured, these concepts

are crucial for handling data in a format that can be easily sent over the network and converted back into usable objects. **Serialization: Definition:**

- Serialization is the process of converting a Java object into a

format (e.g. JSON) that can be easily transmitted over a network or save to a file. **Purpose:**

- Prepare data for transport or storage in structured format.
- When making a request to a REST API, data needs to be sent in a format

that the server can understand.

- Serialization converts Java objects into formats like JSON/XML, which

are commonly used in API communication. **Ease of Transport:**

- Serialized data is often more compact and efficient for transport.

It reduces the amount of data sent over the network, improving performance. **Interoperability:**

- Serialization allows for Interoperability between systems using different

programming languages. JSON, for example, is a language-agnostic format commonly used for Serialization. **In Rest-Assured:**

- Used when sending a request, converting a POJO to JSON to include

it in the request body. **Deserialization: Definition:**

- Deserialization is the process of converting data in a structured

format (e.g. JSON) to Java objects.

- Means, converting data from its Serialized format back into a

usable for (e.g. Java objects) **Purpose:**

- Reconstruct Java objects from received data for easy manipulation

and analysis. **API Response Handling:**

- When an API responds, the data is typically serialized (e.g. JSON).

Deserialization is necessary to convert this serialized data into usable Java objects for further analysis and manipulations. **Data Extraction:**

- Deserialization simplifies the extraction of specific information

from a response. Instead of manually parsing raw data, you work with familiar Java objects. **Object Manipulation:**

- Deserialization allows you to manipulate and work with the data in its original meaningful form. You can use the response data as Java objects in your application. **In Rest-Assured:**

Used when receiving a response, converting JSON response body to a POJO for easy extraction of data. In simple terms:

Serialization: Object -- JSON/XML

Deserialization: JSON/XML – Object

Use cases in Rest-Assured:

Request Payload:

Serialization is used when constructing request payload in Rest-Assured. You convert Java objects(e.g. POJOs) into JSON/XML to include in the body of POST or PUT requests.

Response Handling: Deserialization is used when handling API responses in Rest-Assured. You convert the serialized response body (e.g. JSON) into Java Objects for easy extraction and analysis.

Creating a pojo class for doing the serialization:

```
public class User {  
    private int id;  
    private String name;  
    public User(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    public int getId() {  
        return id;  
    }  
    public void setId(int id) {  
        this.id = id;  
    }  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

Serialization program → Java pojo class to json format

```
public class Serialization_Example {  
  
    @Test  
  
    public void Serializationinrestassured() {  
  
        RestAssured.baseURI="https://jsonplaceholder.typicode.com";  
  
        //Way1  
  
        //Add the gson dependency  
  
        //Create an user pojo class and instance for that  
  
        User user=new User(2,"Vijay");  
  
  
        RequestSpecification reqspec=RestAssured.given();  
  
        Response request=  
reqspec.basePath("/users").contentType(ContentType.JSON).body(user).post();  
  
        System.out.println(request.asPrettyString());  
  
        Response response=reqspec.get("/users/1");  
  
        System.out.println(response.asPrettyString());  
  
        //Way 2  
  
        RestAssured.given()  
  
        .contentType(ContentType.JSON)  
  
        .body(user) //Converts pojo to json  
  
        .when()  
  
        .post("/users").  
  
        then().  
  
        statusCode(201);  
  
    }  
  
}
```

The test will pass.

De-serialization: → Json format to Pojo java class

```
public class Deserializationexample {  
  
    @Test
```



```

public void deserlizeinrestassured() {
    RestAssured.baseURI="https://jsonplaceholder.typicode.com";
    RequestSpecification reqspec= RestAssured.given();
    Response response=reqspec.get("users/1");
    System.out.println(response.getStatusCode());
    User fetcheduser=response.as(User.class); //Converting the Json to java Pojo
    System.out.println("The id of the fetched user "+fetcheduser.getId());
    System.out.println("The name of the fetched user "+fetcheduser.getName());
}
}

```

Get Method :

```

public class Rest1 {
    public static void main(String[] args) {

```

Way 1

```

    RestAssured.baseURI="https://fakestoreapi.com/products";
    RequestSpecification requestspec=RestAssured.given();
    Response response=requestspec.get();
    System.out.println(response.prettyPrint());

```

way2

```

    RestAssured.baseURI="https://fakestoreapi.com/products";
    RequestSpecification reqspec=RestAssured.given();
    Response response=reqspec.request(Method.GET);
    System.out.println(response.prettyPrint());

```

way3

```

    RestAssured.given().when().get("https://fakestoreapi.com/products").then().log().all();
}
}

```

Post Method

Automate POST request using Rest-Assured:

1. Use the resource -- <https://fakestoreapi.com/products>
2. Specify the HTTP method (i.e. POST method) 3. Send the Request to the Server.

- Different ways to construct the request body

4. Get the Response from the Server. 5. Print the response. POST Request:

- POST request is one of the HTTP Methods.
- It is used to submit data to be processed to a specified resource.
- POST request is often used to create a new resource on the server.
- POST requests can include a request body that contains data to be sent to the server. This data can be in various formats such as JSON, XML or form parameters.

[Json data to be input for sending the post request](#)

```
{ "title": "test product",  
  "price": 13.9,  
  "description": "Loharam ipsum set",  
  "category": "electronics"}
```

There are 3 ways for posting the data into the Api they are

1>[Json request body as string](#)

@Test

```
public void addproductusingJSONAsString() {
```

```
String JsonrequestbodyasString="{\"title\": \"Laptop product\", \"price\": 13.9}";
```

```
    //Way 1
```

```
    RestAssured.baseURI="https://fakestoreapi.com/products";
```

```
    RequestSpecification reqspec=RestAssured.given();
```

```
    reqspec.contentType(ContentType.JSON);
```

```
    reqspec.body(JsonrequestbodyasString);
```

```
    Response response=reqspec.post();
```

```
    System.out.println(response.getBody().asPrettyString());
```

```
    //Way2
```

```
    RestAssured.given()
```

```
        .contentType(ContentType.JSON)
```

```
        .body(JsonrequestbodyasString)
```

```

        .post("https://fakestoreapi.com/products")
        .then().log().all();
    }

```

2>Json request body as Map

```

    @Test
    public void addProductusingMap()
    {
        //Gson dependency need to be added

        HashMap<String,String> jsonrequestbodyasmap=new HashMap<>();
        jsonrequestbodyasmap.put("title", "Laptop");
        jsonrequestbodyasmap.put("price", "200");

        //Way1

        RestAssured.baseURI="https://fakestoreapi.com/products";
        RequestSpecification reqspec=RestAssured.given();
        reqspec.contentType(ContentType.JSON);
        reqspec.body(jsonrequestbodyasmap);
        Response response=reqspec.post();
        System.out.println(response.getStatusCode());
        System.out.println(response.getBody().asPrettyString());

        //Way2

        RestAssured.given().contentType(ContentType.JSON)
        .body(jsonrequestbodyasmap).when()
        .post("https://fakestoreapi.com/products")
        .then()
        .log().all();
    }

```

3>Sending post request through the json Object

```

    @Test
    public void addProductUsingJsonObject() {
        //Add the Json dependency in the pom.xml-->import the Org.json not the others

        JSONObject jsonrequestbody=new JSONObject();

```

```

        jsonrequestbody.put("title", "Mobile");
        jsonrequestbody.put("price", "300");

        //Way1

        RestAssured.baseURI="https://fakestoreapi.com/products";
        RequestSpecification reqspec=RestAssured.given();
        reqspec.contentType(ContentType.JSON);
        reqspec.body(jsonrequestbody.toString()); //Here we need to send the data as string
        hence to string method used since json obj wont get paused.

        Response response=reqspec.post();

        System.out.println(response.getStatusCode());
        System.out.println(response.getBody().asPrettyString());

        //Way2

        RestAssured.given().contentType(ContentType.JSON).body(jsonrequestbody.toString())

            .when().post("https://fakestoreapi.com/products")

            .then().log().all();

    }
}

```

4>Json body as the POJO class

//Create one pkg and class==>[testdata](#) and PojoProductrequest

```

package Testdata;

public class PojoProductrequest {

    private String title;

    private int price;

    public PojoProductrequest(String title, int price)

        this.title = title;

        this.price = price;

    }

    public String getTitle() {

        return title;

    }

    public void setTitle(String title) {

```

```

        this.title = title;
    }

    public int getPrice() {
        return price;
    }

    public void setPrice(int price) {
        this.price = price;
    }
}

```

//Write the code in the testing class as follows

```

public void addproductUsingPOJOClass() {

    //Create one pkg and class==>testdata and PojoProductrequest
    PojoProductrequest pojorequestbody=new PojoProductrequest("Bulb", 90);

    //U may use the setter methods as well
    //pojorequestbody.setTitle("Bulb");
    //pojorequestbody.setPrice(90);

    //Way1
    RestAssured.baseURI="https://fakestoreapi.com/products";

    RequestSpecification reqspec=RestAssured.given();

    reqspec.contentType(ContentType.JSON);

    reqspec.body(pojorequestbody);

    Response response=reqspec.post();

    System.out.println(response.getStatusCode());

    System.out.println(response.getBody().asPrettyString());

    //Way2

    RestAssured.given().contentType(ContentType.JSON).body(pojorequestbody)

    .when().post("https://fakestoreapi.com/products")

    .then().log().all();

}

```

5>Json body from the external source

→Create 1 json file in the same project as Product.json and include the json data as

```
{
  "title": "test product",
    "price": 13.9,
    "description": "Loharam ipsum set",
    "category": "electronics"
}
```

Write the code in the test class as follows:

@Test

```
    public void addproductusingexternalJsonFile() {

        //Need to create one json file==>Created Product.json file in the same project and
        given the input data

        File jsonexternalsource=new File("./Product.json");

        //Way1

        RestAssured.baseURI="https://fakestoreapi.com/products";
        RequestSpecification reqspec=RestAssured.given();
        reqspec.contentType(ContentType.JSON);
        reqspec.body(jsonexternalsource);
        Response response=reqspec.post();
        System.out.println(response.getStatusCode());
        System.out.println(response.getBody().asPrettyString());

        //Way2

        RestAssured.given().contentType(ContentType.JSON)
        .body(jsonexternalsource).when()
        .post("https://fakestoreapi.com/products")
        .then().log().all();

    }
```

Put Method :

Automate PUT request using Rest-Assured:

1. Use the resource
2. Specify the HTTP method (i.e. PUT method)
3. Send the Request to the Server.

4. Get the Response from the Server.

5. Print/Verify the response.

PUT Request:

Use the below url for performing the Put method operation:

- <https://fakestoreapi.com/products/20>==>we are modifying the item at the id=20
- PUT request is one of the HTTP Methods.
- It is used to update or create a resource on the server.
- PUT request is often used to modify the state or update the

data of an existing resource identified by a URI.

- The PUT method in essence ensures that when the same request is

repeated, the outcome remains consistent and identical to that of a single request.

- If the resource identified by the URI does not exist, the

server may create a new resource with that URI.

➔Select the Put method in the postman and use the following code in the raw- body of type json.

```
{  
  "title": "Updated Product Title",  
  "price": 29956789.99  
}
```

And click on the send button you will get the result 200OK and the following

```
{  
  "id": 20,  
  "title": "Updated Product Title",  
  "price": 29956789.99  
}
```

The Put method in the Restassured:

```
public class Putmethodinrestassured {  
    @Test  
    public void putmethodexample() {  
        //Json object is the type of org.json.JSONObject;  
        JSONObject jsonobjectbody=new JSONObject();  
        jsonobjectbody.put("title", "Updated Product Title");
```

```

        jsonobjectbody.put("price", 29956789.99);

        //Way1

        RestAssured.baseURI="https://fakestoreapi.com/products";

        RequestSpecification reqspec=RestAssured.given();

        reqspec.contentType(ContentType.JSON);

        reqspec.body(jsonobjectbody.toString());

        Response response=reqspec.put("/20");

        System.out.println(response.getStatusCode());

        System.out.println(response.body().asPrettyString());

        //Way2

        RestAssured.given().contentType(ContentType.JSON)

        .baseUri("https://fakestoreapi.com/products")

        .body(jsonobjectbody.toString())

        .when().put("/20")

        .then().statusCode(200).log().all();

    }

}

```

Delete Method :

```

public class Deletemethodinrestassured {

    @Test

    public void deletemethodexample() {

        //Way1

        RestAssured.baseURI="https://fakestoreapi.com/products";

        RequestSpecification reqspec=RestAssured.given();

        reqspec.contentType(ContentType.JSON);

        Response response=reqspec.delete("/20");

        System.out.println(response.getStatusCode());

        System.out.println(response.body().asPrettyString());

        //Way2

        RestAssured.given().contentType(ContentType.JSON)

        .baseUri("https://fakestoreapi.com/products")
    }
}

```



```

        .when().delete("/20")

        .then().statusCode(200).log().all();
    }

}

```

Patch Method

Post Request:

Use the below url for performing the Put method operation:

- <https://fakestoreapi.com/products/20>==>we are modifying the item at the id=20
- Patch request is one of the HTTP Methods.
- If the resource identified by the URI does not exist, the

server may create a new resource with that URI.

➔Select the Put method in the postman and use the following code in the raw- body of type json.

```

{
  "title": "Updated Product Title",
  "price": 29956789.99
}

```

And click on the send button you will get the result 200OK and the following

```

{
  "id": 20,
  "title": "Updated Product Title",
  "price": 29956789.99
}

```

The Put method in the Restassured:

```

public class Putmethodinrestassured {

    @Test

    public void putmethodexample() {

        //Json object is the type of org.json.JSONObject;

        JSONObject jsonobjectbody=new JSONObject();

        jsonobjectbody.put("title", "Updated Product Title");
    }
}

```

```

        jsonobjectbody.put("price", 29956789.99);

        //Way1

        RestAssured.baseURI="https://fakestoreapi.com/products";

        RequestSpecification reqspec=RestAssured.given();

        reqspec.contentType(ContentType.JSON);

        reqspec.body(jsonobjectbody.toString());

        Response response=reqspec.patch("/20");

        System.out.println(response.getStatusCode());

        System.out.println(response.body().asPrettyString());

        //Way2

        RestAssured.given().contentType(ContentType.JSON)

        .baseUri("https://fakestoreapi.com/products")

        .body(jsonobjectbody.toString())

        .when().patch("/20")

        .then().statusCode(200).log().all();

    }

}

```

Path Parameters in Rest Assured:

- Path parameters are part of the URL path itself.
- They are used to identify a specific resource.
- Path parameters are typically used for operations like fetching a specific resource by its unique identifier.

Use the Url → <https://fakestoreapi.com/products/:id>

In the above url **:id** → path parameter => use id less than 20.

Copy paste the link in the postman and under the params below the path Variables provide the output for the Id based on that the data will be fetched.

Path parameters in rest-assured:

```

public class Pathparameters {

    @Test

    public void Pathparameterex() {

        //Way1
    }
}

```

```

RestAssured.baseURI="https://fakestoreapi.com";

RequestSpecification reqspec=RestAssured.given();

reqspec.pathParam("id", "3");

Response res=reqspec.get("/products/{id}");

System.out.println(res.statusCode());

System.out.println(res.getBody().asPrettyString());

```

```

//Way2

RestAssured

.given()

.pathParam("id", 10)

.when().get("https://fakestoreapi.com/products/{id}")

.then()

.log()

.all();

```

```

}

```

```

}

```

Query parameters:

- Use the Url→<https://www.google.com/search?q=rose>
- The Query parameter starts with the ?==>In the above example “?q=rose”

Query Parameters in Rest Assured:

- Query parameters are appended to the URL after a question mark (?)

and separated by ampersands (&).

- They are used for filtering, sorting, or specifying additional

information for a request.

- Query parameters are optional and can be used for operations that don't necessarily identify a specific resource.

Query parameters in the Rest-assured:

```

public class Pathparameters {

    @Test

    public void Queryparameters() {

        //Way1

```

```

RestAssured.baseURI="https://www.google.com/search";

RequestSpecification reqspec=RestAssured.given();

reqspec.queryParam("q", "rose");

Response res=reqspec.get();

System.out.println(res.statusCode());

//System.out.println(res.getBody().asPrettyString());

//Way2

RestAssured

.given()

.queryParam("q", "school")

.when().get("https://www.google.com/search")

.then()

.log()

.all();
}
}

```

JSON Schema:

JSON Schema is like a blueprint for defining the structure and rules of JSON documents. Example: Consider a JSON Schema that describes the structure of A person's information

```

{ "type": "object", "properties": {
  "name": { "type": "string",
  "age": { "type": "integer",
  "email": { "type": "string", "format": "email" } },
  "required": [ "name", "age" ] }

```

This schema defines that a person object must have name (string) and age (integer), and optionally an email (string in email format).

Purpose of JSON Schema:

JSON schema acts as a contract, ensuring that JSON data conforms to predefined rules. Example:

With the previous schema, if someone sends data like: { "name": "Tom", "age": 30, "email": "tom@gmail.com" } It conforms to the schema since it includes name and age, and email is optional. It helps maintain data integrity and consistency by enforcing rules of JSON data.

Example: If someone send invalid data like:

```
{ "name": "jerry",  
  "age": "twenty-five" }
```

It violates the schema since age should be an integer, enforcing consistency in data types. Usefulness of JSON Schema:

- JSON schema enables automatic validation of JSON data against predefined rules.
- Ex: When receiving JSON data from a client, you can automatically validate it against the schema to ensure it meets the expected structure and constraints.
- It improves Interoperability by providing a standard format for JSON data exchange.

Ex: Different systems can agree on a common JSON schema for data exchange, ensuring compatibility and seamless communication between them.

- JSON schema enhances development by serving as a blueprint for designing JSON APIs and data models.

Ex: When designing an API, defining JSON schema upfront helps in clarifying data requirements and facilitates smoother development and integration.

- It can be used in various contexts, including API documentation, web application data validation, and data interchange formats.
- Ex: Including JSON schema in API documentation helps developers understand the expected format of request and response payloads, reducing ambiguity and errors.

JSON Schema Generator -- <https://transform.tools/json-to-json-schema>

JSON Schema Validator -- <https://jsonschemalint.com/#!/version/2020-12/markup/json>

JSON Formatter -- <https://jsonformatter.org/>

JSON Formatter – Used for the converting the unstructured json data into a formatted json data and also corrects if any mistakes in the json data.

JSON Schema Generator –

Used for generating the schema for the json data that we have pasted from the output

Video → Krishna sakinala → FRestassured → 26 video after the 16min

Json Schema validator:

→ Create a new file name in the src/test/resources, it as Fakeapi. Json and provide the schema from which is generated from the JSON Schema Generator

→ Use the Url → <https://fakestoreapi.com/products>

And the o/p needs to be given for the JSON Schema Generator for getting the Schema header and put that generated schema header in the Fakeapi.json.

Ex: After generating the header schema

```
{
```

```
"$schema": "http://json-schema.org/draft-07/schema#",
"type": "array",
"items": {
  "type": "object",
  "required": ["id", "title", "price", "description", "category", "image", "rating"],
  "properties": {
    "id": {
      "type": "integer"
    },
    "title": {
      "type": "string"
    },
    "price": {
      "type": "number"
    },
    "description": {
      "type": "string"
    },
    "category": {
      "type": "string"
    },
    "image": {
      "type": "string"
    },
    "rating": {
      "type": "object",
      "required": ["rate", "count"],
      "properties": {
        "rate": {
          "type": "number"
        }
      }
    }
  }
}
```

```

    "count": {
        "type": "integer"
    }
}
}
}
}
}
}
}
}
}

```

Schema validator in the rest assured:

```

import static io.restassured.RestAssured.*;
import static io.restassured.module.json.JsonSchemaValidator.*;

public class JsonValidator {

    @Test

    public void Jsonvalidator() {

        //Need to add the rest-assured Json Schema validator dependency
        //Way1
        RestAssured.baseURI = "https://fakestoreapi.com/products";
        RequestSpecification reqspec = RestAssured.given();
        Response res = reqspec.get();
        res.then()
            .statusCode(200)
            .body(matchesJsonSchemaInClasspath("FakeApi.json"));
        System.out.println(res.statusCode());
        //Way2
        RestAssured
            .given()
            .when().get("https://fakestoreapi.com/products")
            .then()
            .assertThat()
            .body(JsonSchemaValidator.matchesJsonSchemaInClasspath("FakeApi.json"));
    }
}

```

